THREADED PROGRAMMING

GIOVANNA LEHMANN MIOTTO

ISOTDAQ SCHOOL 2012

OUTLINE

- **Multi-tasking, concurrency**
- **A bit of history**
- **What is a thread**
- **Essentials in concurrent programming (Java)**
  - Safety, liveness, performance
  - Memory model
- **Conclusions**
- **References**

MULTI-TASKING

- **We are used to it in every day life**
  - Watch TV while eating pop-corns and caressing your cat
  - Send an e-mail, launch a job on the printer and do something else while waiting for the answer and the printed document
  - …
- **On a Computer**
  - While a task waits on input, another one performs calculations on some data and a third one outputs messages

MULTI-TASKING

- **There are two different types of multi-tasking**
  - Interleaved usage of one resource by different tasks
    - I can use the same hand to caress my cat and eat pop-corn, but in reality I'll use the hand in a sequential way to either caress the cat or put the pop-corn in my mouth
  - Parallel usage of the same type of resource by different tasks
    - Using two hands the action of caressing the cat and putting pop-corn in my mouth can happen AT THE SAME TIME
    - both hands are capable of doing both tasks

EVOLUTION OF COMPUTER MULTI-TASKING

Multi-tasking history starts with the introduction of operating systems -> task scheduler

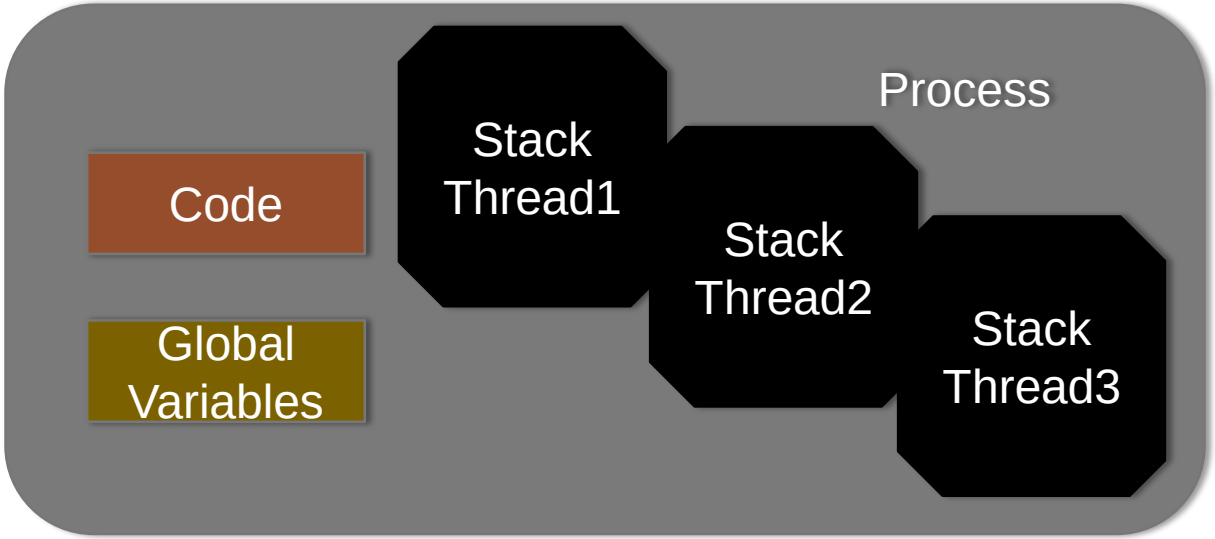
- **<u>Multiprogramming</u>**
  Program runs until it reaches instruction waiting for a peripheral, then its context is stored away and another program starts
- **<u>Cooperative multitasking</u>**
  Programs voluntarily cede time to one another
- **<u>Preemptive multitasking</u>**
  Operating system guarantees that each program gets time for execution + handling of interrupts (I/O)

FROM PROCESSES TO THREADS

- **Multitasking improved throughput of computers**
- **Developers started developing applications as sets of cooperating processes (e.g. one gets the input data, another performs calculations on the data, a third writes out the results) -> need for sharing data**
- **Threads born from the idea that most efficient way of sharing data was to share entire memory space**

WHAT IS A THREAD

- **Smallest unit of processing that can be scheduled by an operating system**
- **Threads are contained inside processes**
- **Threads of a process share memory and other resources**

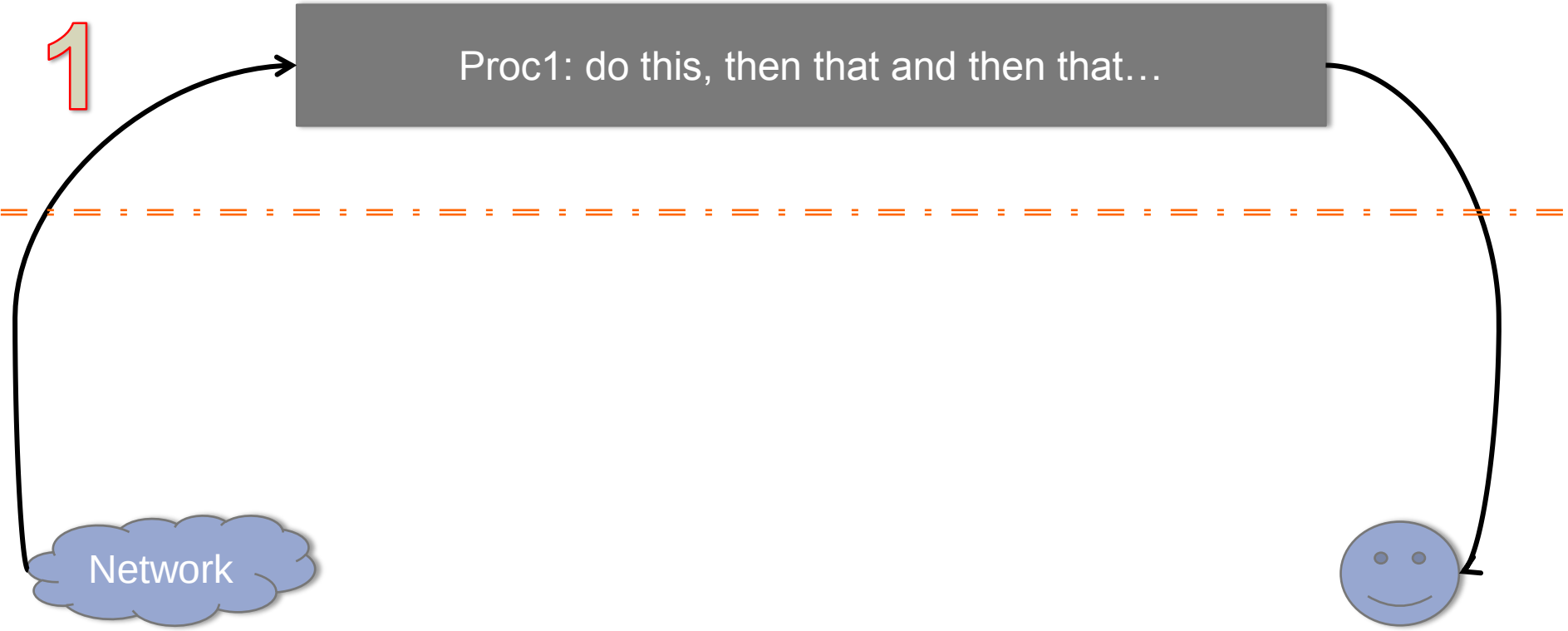


PROCESS VS THREAD

Each *process* provides the resources needed to execute a program. A process has a virtual address space, executable code, open handles to system objects, a security context, a unique process identifier, environment variables, a priority class, minimum and maximum working set sizes, and at least one thread of execution. Each process is started with a single thread, often called the *primary thread*, but can create additional threads from any of its threads.

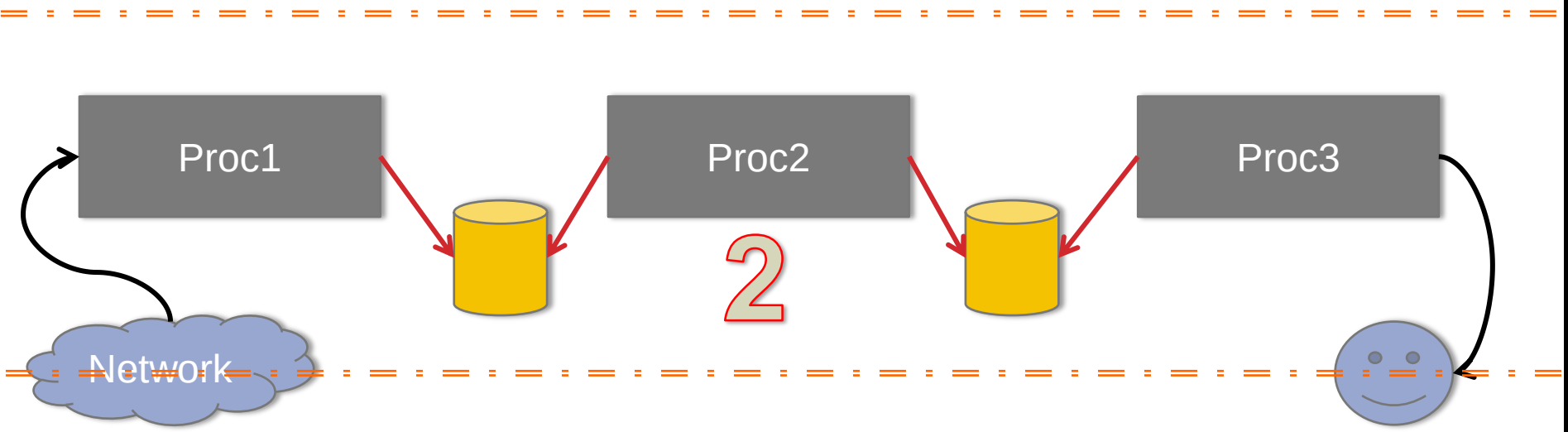
A *thread* is the entity within a process that can be scheduled for execution. All threads of a process share its virtual address space and system resources. In addition, each thread maintains exception handlers, a scheduling priority, thread local storage, a unique thread identifier, and a set of structures the system will use to save the thread context until it is scheduled. The *thread context* includes the thread's set of machine registers, the kernel stack, a thread environment block, and a user stack in the address space of the thread's process. Threads can also have their own security context, which can be used for impersonating clients.

SW DESIGN EVOLUTION



- **Difficult to handle "de-randomization"….**
- **Code filled with loops and checks**

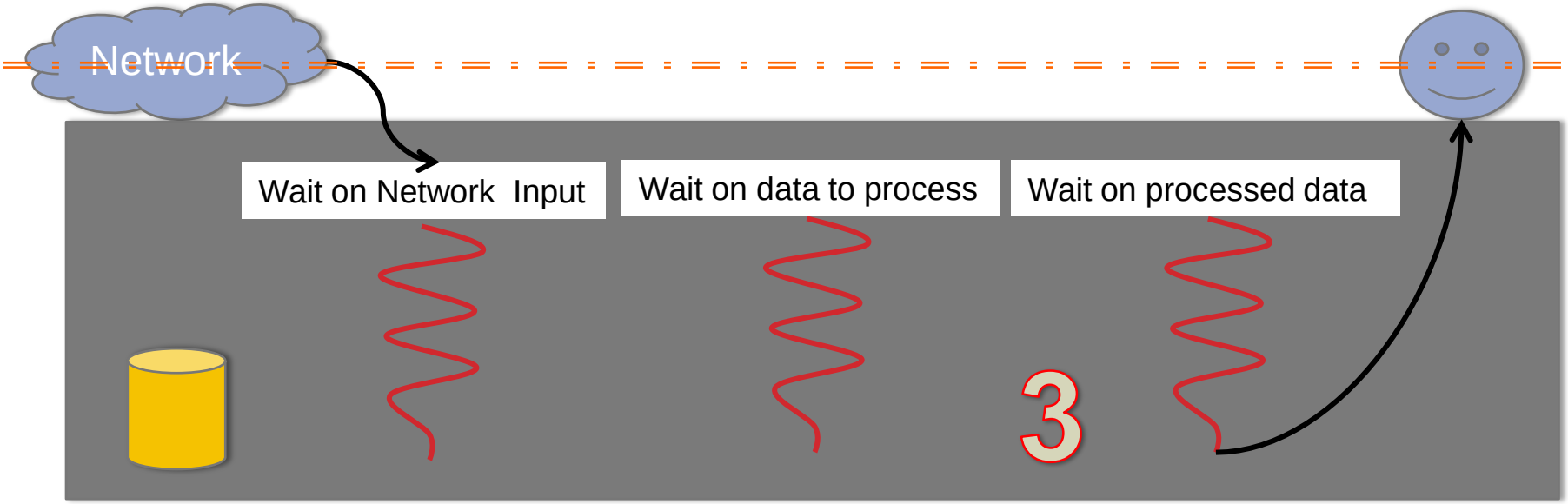
SW DESIGN EVOLUTION



- **Code better structured, cleaner**
- **Many data copies**

SW DESIGN EVOLUTION

- **Preserve code simplicity**
- **Optimize usage of resources (memory!)**



MULTI-THREADING

There are two different environments for multi-threading

- **Single processor: Interleaved usage of one resource by different tasks**
  - Actions “seem” to happen in parallel but they are actually sequential
  - Multithreading allows to design simpler, modular code with efficient use of resources
- **Multiple processors/cores: Parallel usage of the same type of resource by different tasks**
  - Threads are truly running AT THE SAME TIME
  - Multithreading allows the usage of multiple CPUs by one process!

PROS AND CONS OF THREADS

+

- Exploiting multiple processors
- Simplicity of modeling
- Simplified handling of asynchronous commands
- More responsive user interfaces

-

- Safety hazards
- Liveness hazards
- Performance hazards

"Although threads seem to be a small step from sequential computation, in fact, they represent a huge step. They discard the most essential and appealing properties of sequential computation: understandability, predictability, and determinism. Threads, as a model of computation, are wildly nondeterministic, and the job of the programmer becomes one of pruning that non-determinism." -- 'The Problem with Threads, Edward A. Lee, UC Berkeley, 2006

THREAD SAFETY

- **Managing access to state and, in particular, to shared, mutable state.**
- **State: any data that can affect externally visible behavior of an object**
- **Shared: variable that can be accessed by several threads**
- **Mutable: value of a variable can change over time**

- **Writing thread safe code is about protecting data from uncontrolled concurrent access**

THREAD SAFETY

If different threads access the same mutable state variable without appropriate synchronization <u>your program is broken</u>.

To fix it:

- **Don't share state variables across threads**
- **Make state variables immutable**
- **Use synchronization whenever accessing the variable**

It is far easier to design a class to be thread-safe than to retrofit it for thread safety later.